

Name: Sachin
Roll. No.: 251256
Dept: B.sc (H.) CS
Data mining (DM) ASSIGNMENT: 3

1. Apply data cleaning techniques on any dataset (e.g., Paper Reviews dataset in UCI repository). Techniques may include handling missing values, outliers and inconsistent values. A set of validation rules can be prepared based on the dataset and validations can be performed.

```

Q1.py > ...
1  file = open("paper_reviews.csv", "r")
2  lines = file.readlines()
3  file.close()
4
5  data = []
6
7  for line in lines[1:]:
8      data.append(line.strip().split(","))
9
10 print("Original Data:")
11 for row in data:
12     print(row)
13
14 # Missing values
15 for row in data:
16     for i in range(1, 4):
17         if row[i] == "":
18             row[i] = "3"
19
20 # Inconsistent values
21 for row in data:
22     row[0] = row[0].strip().lower()
23     row[4] = row[4].strip().lower()
24
25 # Remove duplicates
26 cleaned = []
27
28 for row in data:
29     if row not in cleaned:
30         cleaned.append(row)
31
32 data = cleaned
33

```

```

34 # Handle outliers
35 for row in data:
36     for i in range(1, 4):
37         if float(row[i]) < 1 or float(row[i]) > 5:
38             row[i] = "5"
39
40 # Validation
41 missing = 0
42 duplicates = 0
43 invalid = 0
44
45 for row in data:
46     for value in row:
47         if value == "":
48             missing += 1
49
50     for value in row[1:4]:
51         if float(value) < 1 or float(value) > 5:
52             invalid += 1
53
54 print("\nValidation:")
55 print("Missing values:", missing)
56 print("Duplicate rows:", duplicates)
57 print("Invalid values:", invalid)
58
59 print("\nCleaned Data:")
60 for row in data:
61     print(row)
62
63 # Save cleaned data
64 file = open("cleaned_paper_reviews.csv", "w")
65
66 file.write("reviewer,originality,quality,clarity,decision\n")
67
68 for row in data:
69     file.write(",".join(row) + "\n")

```

```

70
71 file.close()
72
73 print("\nCleaned dataset saved successfully.")

```

Output:

```
(.venv) PS D:\DM assignment 1> python Q1.py
Original Data:
['Alice', '4', '5', '4', 'accept']
['Bob', '3', '4', '3', 'reject']
['Charlie', '5', '5', '4', 'accept']
['David', '', '3', '4', 'reject']
['Eve', '4', '4', '5', 'accept']
['Frank', '2', '3', '2', 'reject']
['Alice', '4', '5', '4', 'accept']
['George', '50', '4', '3', 'accept']
['Harry', '3', '', '4', 'reject']

Validation:
Missing values: 0
Duplicate rows: 0
Invalid values: 0

Cleaned Data:
['alice', '4', '5', '4', 'accept']
['bob', '3', '4', '3', 'reject']
['charlie', '5', '5', '4', 'accept']
['david', '3', '3', '4', 'reject']
['eve', '4', '4', '5', 'accept']
['frank', '2', '3', '2', 'reject']
['george', '5', '4', '3', 'accept']
['harry', '3', '3', '4', 'reject']

Cleaned dataset saved successfully.
```

Original data:

	A	B	C	D	E	F
1	reviewer	originality	quality	clarity	decision	
2	Alice	4	5	4	accept	
3	Bob	3	4	3	reject	
4	Charlie	5	5	4	accept	
5	David		3	4	reject	
6	Eve	4	4	5	accept	
7	Frank	2	3	2	reject	
8	Alice	4	5	4	accept	
9	George	50	4	3	accept	
10	Harry	3		4	reject	

Cleaned data:

	A	B	C	D	E	F
1	reviewer	originality	quality	clarity	decision	
2	alice	4	5	4	accept	
3	bob	3	4	3	reject	
4	charlie	5	5	4	accept	
5	david	3	3	4	reject	
6	eve	4	4	5	accept	
7	frank	2	3	2	reject	
8	george	5	4	3	accept	
9	harry	3	3	4	reject	
10						

2. Apply data pre-processing techniques such as standardization/normalization, transformation, aggregation, discretization/binarization, sampling etc. on any dataset

```
DM-Assignment2.py > ...
1  students = [
2      {"id": 1, "name": "Shyam", "marks": 85, "attendance": 92},
3      {"id": 2, "name": "Priya", "marks": 72, "attendance": 85},
4      {"id": 3, "name": "mukul", "marks": 55, "attendance": 68},
5      {"id": 4, "name": "Nehal", "marks": 91, "attendance": 95},
6      {"id": 5, "name": "Ravin", "marks": 45, "attendance": 60},
7      {"id": 6, "name": "Sneha", "marks": 78, "attendance": 88},
8      {"id": 7, "name": "Karam", "marks": 62, "attendance": 75},
9      {"id": 8, "name": "Akash", "marks": 39, "attendance": 55}
10 ]
11
12
13 # Original Data
14 print("----- STUDENT DATA -----")
15 print("ID", "Name", "Marks", "Attendance")
16
17 for s in students:
18     print(s["id"], s["name"], s["marks"], s["attendance"])
19
20
21 # 1. NORMALIZATION - Min-Max
22 all_marks = [s["marks"] for s in students]
23
24 low = min(all_marks)
25 high = max(all_marks)
26
27 print("\n----- NORMALIZATION -----")
28
29 for s in students:
30     value = (s["marks"] - low) / (high - low)
31     print(s["name"], "=>", round(value, 2))
32
```

```

34 # 2. STANDARDIZATION - Z-Score
35 mean = sum(all_marks) / len(all_marks)
36
37 variance = sum((x - mean) ** 2 for x in all_marks) / len(all_marks)
38 std = variance ** 0.5
39
40 print("\n----- STANDARDIZATION -----")
41
42 for s in students:
43     value = (s["marks"] - mean) / std
44     print(s["name"], "=>", round(value, 2))
45
46
47 # 3. TRANSFORMATION
48 print("\n----- TRANSFORMATION -----")
49
50 for s in students:
51     percentage = (s["marks"] / 100) * 100
52     print(s["name"], "=>", percentage, "%")
53
54
55 # 4. AGGREGATION
56 sum_marks = 0
57
58 for s in students:
59     sum_marks += s["marks"]
60
61 avg_marks = sum_marks / len(students)
62
63 print("\n----- AGGREGATION -----")
64 print("Total Marks:", sum_marks)
65 print("Average Marks:", round(avg_marks, 2))
66
```

```

68 # 5. DISCRETIZATION
69 print("\n----- DISCRETIZATION -----")
70
71 for s in students:
72     score = s["marks"]
73
74     if score < 50:
75         level = "Poor"
76     elif score < 75:
77         level = "Average"
78     else:
79         level = "Good"
80
81     print(s["name"], "=", level)
82
83
84 # 6. BINARIZATION
85 print("\n----- BINARIZATION -----")
86
87 for s in students:
88     if s["marks"] >= 50:
89         status = 1
90     else:
91         status = 0
92
93     print(s["name"], "=", status)
94
95
96 # 7. SAMPLING
97 print("\n----- SAMPLING -----")
98 print("Alternate Records:")
99
100 for i in range(0, len(students), 2):
101     print(students[i])

```

Output:

```
----- STUDENT DATA -----
ID Name Marks Attendance
1 Shyam 85 92
2 Priya 72 85
3 mukul 55 68
4 Nehal 91 95
5 Ravin 45 60
6 Sneha 78 88
7 Karam 62 75
8 Akash 39 55
```

```
----- NORMALIZATION -----
Shyam => 0.88
Priya => 0.63
mukul => 0.31
Nehal => 1.0
Ravin => 0.12
Sneha => 0.75
Karam => 0.44
Akash => 0.0
```

```
----- STANDARDIZATION -----
Shyam => 1.09
Priya => 0.35
mukul => -0.62
Nehal => 1.43
Ravin => -1.19
Sneha => 0.69
Karam => -0.22
Akash => -1.53
```

```
----- TRANSFORMATION -----
Shyam => 85.0 %
Priya => 72.0 %
mukul => 55.00000000000001 %
Nehal => 91.0 %
Ravin => 45.0 %
Sneha => 78.0 %
Karam => 62.0 %
Akash => 39.0 %
```

```
----- AGGREGATION -----
Total Marks: 527
Average Marks: 65.88
```

```
----- DISCRETIZATION -----
Shyam => Good
Priya => Average
mukul => Average
Nehal => Good
Ravin => Poor
Sneha => Good
Karam => Average
Akash => Poor
```

```
----- BINARIZATION -----
Shyam => 1
Priya => 1
mukul => 1
Nehal => 1
Ravin => 0
Sneha => 1
Karam => 1
Akash => 0
```

```
----- SAMPLING -----
Alternate Records:
{'id': 1, 'name': 'Shyam', 'marks': 85, 'attendance': 92}
{'id': 3, 'name': 'mukul', 'marks': 55, 'attendance': 68}
{'id': 5, 'name': 'Ravin', 'marks': 45, 'attendance': 60}
```

Thank You